Optimisations du jeu VR

Les 8 optimisations principales : ce qui a été fait, où, et comment le vérifier

Projet	SAÉ 5D.01 · tower defense VR · BUT MMI 3, IUT Béziers
Dans le code	branche `main` (PR #188 fusionnée le 10 nov. 2026)
Par	Dylan, avec Claude (Claude Code), usage noté dans `docs/journal/dylan.md`
Objectif	tenir 72 images par seconde sur le Meta Quest

#	Optimisation	En une phrase	Page
1	LOD sur les modèles	Unity calcule des versions simplifiées des modèles et prend la plus légère selon la distance.	2
2	Instancing et ballons allégés	Les ballons du plateau sont dessinés par couleur en un seul appel, avec une boule de 80 faces.	2
3	Matériaux partagés	Un seul matériau par couleur, pour que le casque regroupe les objets.	3
4	Ombres allégées	Ombres à 15 m, et plus d'ombre pour les objets petits, nombreux ou rapides.	3
5	Décor static	Le décor immobile est marqué static : Unity le regroupe au lieu de le dessiner morceau par morceau.	3
6	Pool de ballons	Les ballons éclatés sont rangés et réutilisés au lieu d'être détruits et recréés.	4
7	Rigidbody kinematic	Les ballons ont un Rigidbody kinematic : la physique ne recalcule plus tout quand ils bougent.	4
8	Singes allégés	Singes figés ou immobiles quand on ne les voit pas, ciblage moins fréquent, aura sans particules.	5

À savoir. Tout ce qui suit est dans le code de `main` et compile, mais **rien n'a encore été mesuré au casque**. Chiffres « avant » connus (issue #153, mode admin, 200 singes, dans l'éditeur) : **41 fps, 626 000 triangles, 141 SetPass calls**. Le tableau de la dernière page sert à noter les mesures « après ».

Les chemins de scripts partent de `Unity/Assets/_Project/Scripts/`, noté `Scripts/`. Les numéros de ligne sont ceux de `main` au 10 nov.

1. LOD sur les modèles #176

Ce qui a été fait	Dans Unity 6, l'import d'un modèle a une case **Generate Mesh LODs**. Elle est cochée sur les modèles sans squelette. Unity calcule alors tout seul des versions simplifiées, rangées dans le même maillage. La carte graphique prend la plus légère qui suffit, selon la taille de l'objet à l'écran. C'est surtout utile pour les ballons : un ballon fait 1 032 triangles (1 680 pour le cœur), et il y en a des centaines par vague, souvent loin du joueur.
Où	• 8 fichiers `.fbx.meta` avec `generateMeshLods: 1` : `Art/Ballons/FBX/Ballon_Normal`, `Ballon_Blindage`, `Ballon_Coeur`, `Art/MOAB/FBX/MOAB`, `Art/BFB/FBX/BFB`, `Art/Bananier/FBX/Bananier`, `Art/Canon/FBX/Canon`, `Art/Tireur/FBX/Tireur` • pas de réexport depuis Blender : il suffit de décocher la case pour revenir en arrière
Comment vérifier	• Dans la fenêtre Project, sélectionner `Ballon_Normal.fbx`, puis dans l'Inspector, onglet **Model** : « Generate Mesh LODs » est coché. • En Play, sur la carte pendant une vague : ouvrir **Stats** (en haut à droite de la vue Game) et noter **Tris** en regardant les ballons de près, puis de loin. Le chiffre doit baisser de loin.
Ce qu'on perd	rien de visible : les versions simplifiées ne servent que de loin.

2. Instancing et réduction de polygones des ballons #153

Ce qui a été fait	Le plateau du hub montre la carte en miniature, en direct. Avant, chaque ballon avait sa miniature, un vrai objet avec son modèle complet. Maintenant, les miniatures ne sont plus des objets. À chaque image, toutes les boules d'une même couleur sont dessinées en **un seul appel** (`Graphics.RenderMeshInstanced`) : 500 ballons, c'est quelques appels au lieu de 500. Chaque miniature est une **petite boule de 80 faces**, au lieu d'un modèle de plus de 1 000 triangles. Les taches des ballons camo utilisent la même boule, fusionnée en un seul maillage. **Exception** : les boss et les dirigeables, peu nombreux, gardent une vraie copie de leur modèle pour qu'on les reconnaisse.
Où	• `Scripts/Hub/Board.cs:104` : l'appel instancié, une fois par couleur • `Scripts/Hub/Board.cs:108` : `HasOwnModel` (boss et dirigeables) ; `Miniature` ligne 123 • `Scripts/Core/MeshBatch.cs:65` : `SmallBall`, la boule de 80 faces, créée une seule fois • `Scripts/Map/Balloon.cs:290` : les taches du camo, faites de cette boule
Comment vérifier	• Lancer une vague, revenir au hub, ouvrir **Window** → **Analysis** → **Frame Debugger** → Enable : on doit trouver un appel instancié avec beaucoup d'instances, une par ballon de cette couleur. • **Stats** : le nombre de Batches bouge peu quand le nombre de ballons augmente. • Pendant une vague de boss : sur le plateau, le boss garde son vrai modèle (pas une boule étirée).
Ce qu'on perd	sur le plateau, les ballons normaux sont des boules simples. Ils font quelques centimètres, ça ne se voit pas.

3. Matériaux partagés #153, corrigé le 10 nov.

Ce qui a été fait	Avant, chaque objet coloré recevait sa couleur par un « bloc de propriétés ». Ça empêche le SRP Batcher d'URP de regrouper les objets, donc chaque cube coûtait un changement de matériau. Maintenant, **tous les objets d'une même couleur partagent le même matériau**, créé une seule fois et gardé dans un cache. Pareil pour les singes teintés et les textures des ballons. Les couleurs qui changent à chaque image (cadre du tutoriel qui pulse, boutons, panneau d'argent, roulette, nuages) passent par `Animate` et ne créent aucun matériau. C'était le bug corrigé après la relecture : avant, elles créaient environ 72 matériaux par seconde.
Où	• `Scripts/Core/ColorTint.cs:36` : `Shared`, le cache « un matériau par couleur » • `Scripts/Core/ColorTint.cs` : `Set` (couleur fixe, matériau partagé) et `Animate` (couleur animée, sans matériau) • `Scripts/Core/MonkeyView.cs:106` : les singes teintés ; `Scripts/Core/BalloonVisuals.cs` : les ballons
Comment vérifier	• **Frame Debugger** : les objets de même couleur sont regroupés dans un même « SRP Batch ». • **Stats** : moins de SetPass calls au hub qu'avant (141 dans #153). • Laisser tourner le tutoriel 2-3 minutes, puis **Profiler** → **Memory** : le nombre de matériaux reste stable.

4. Ombres allégées #153, #177

Ce qui a été fait	Chaque objet qui fait une ombre est dessiné une deuxième fois, pour la carte des ombres. Deux changements : les ombres s'arrêtent à **15 m** du joueur, et les objets petits, nombreux ou rapides ne font plus d'ombre du tout. Ça concerne les ballons, les taches du camo, les projectiles, les traits du Sniper, les miniatures du plateau, l'aura, la pluie et les nuages.
Où	• `Unity/Assets/Settings/Mobile_RPAsset.asset:57` et `PC_RPAsset.asset:57` : `m_ShadowDistance: 15` • `ShadowCastingMode.Off` : `Balloon.cs:139` (modèles), `WaveSpawner.cs:191` (sphères), `Projectile.cs:72`, `Tower.cs:119`, `Board.cs:101`, `Aura.cs:64`, `Weather.cs:273`
Comment vérifier	• Sélectionner `Mobile_RPAsset` dans `Assets/Settings` → Shadows → **Max Distance = 15**. • En Play, sélectionner un ballon ou un projectile : Mesh Renderer → Lighting → **Cast Shadows = Off**.
Ce qu'on perd	plus d'ombre au sol sous les ballons, et pas d'ombre au-delà de 15 m.

5. Décor static #153

Ce qui a été fait	Tout le décor qui ne bouge jamais (la cabane, les arbres, les rochers, le décor de la carte) est marqué **static** par le générateur de scènes. Unity sait alors qu'il ne bougera pas, et le regroupe pour le dessiner en moins d'appels (static batching). Les petits éléments répétés (cailloux, champignons) sont en plus fusionnés en un seul maillage. Ce qui bouge ou s'anime (lumières qui s'éteignent, portes) est volontairement laissé non static.
Où	• `Scripts/Editor/PrototypeGenerator.cs:945` : `MakeStatic`, appelé pour la cabane (ligne 886), le décor (ligne 963) et les modèles fixes (ligne 1571) • `Scripts/Core/MeshBatch.cs:34` : le décor fusionné est marqué static • `Scripts/Editor/PrototypeGenerator.cs:708` et 1757 : les objets qui bougent, retirés du static
Comment vérifier	• Après **SAE** → **Générer le prototype**, sélectionner un mur de la cabane : la case **Static** (en haut à droite de l'Inspector) est cochée. • **Frame Debugger** au hub : le décor apparaît en « Static Batch » (quelques appels pour toute la cabane).

6. Pool de ballons #153, #174

Ce qui a été fait	Un ballon éclaté ou sorti de la carte n'est plus détruit. Il est caché et rangé dans une pile, puis ressorti pour le prochain ballon de la même sorte. Créer et détruire des centaines d'objets par vague provoquait des à-coups, à cause du ramasse-miettes. Depuis le 10 nov., chaque ballon a aussi un **numéro de vie**, qui change à chaque sortie. Un tir encore en vol vers un ballon éclaté ne suit donc plus ce ballon quand il ressort à l'entrée de la carte.
Où	• `Scripts/Map/WaveSpawner.cs:35` : le pool, une pile par sorte de ballon ; `Recycle` ligne 220 • `Scripts/Map/Balloon.cs:64-67` : `Life` et `StillThere`, utilisés par `Projectile.cs:91`, `Projectile.cs:107` et `Tower.cs:65`
Comment vérifier	• En Play, pendant une vague, regarder la Hierarchy de la scène Labyrinthe : les ballons éclatés deviennent grisés (inactifs) au lieu de disparaître, et leur nombre ne grimpe pas sans fin. • **Profiler** → **CPU** : la colonne GC Alloc reste basse pendant une vague. • Plusieurs singes sur le même passage et une vague serrée : aucune fléchette ne repart vers l'entrée de la carte.

7. Rigidbody kinematic sur les ballons #174

Ce qui a été fait	Chaque ballon avait un collider, mais **pas de Rigidbody**. Pour le moteur physique, c'est un mur « statique ». Or, déplacer un collider statique à chaque image oblige la physique à reconstruire sa structure, des centaines de fois par image pendant une vague. Avec un Rigidbody **kinematic**, c'est notre code qui déplace le ballon, et la physique le suit sans tout recalculer. La gravité est coupée : le ballon reste sur son chemin.
Où	• `Scripts/Map/WaveSpawner.cs:187` : `body.isKinematic = true` (et `useGravity = false`), à la création du ballon
Comment vérifier	• En Play, sélectionner un ballon : composant **Rigidbody**, « Is Kinematic » coché, « Use Gravity » décoché. • **Profiler** → **Physics** : le temps de physique reste faible pendant une grosse vague.

8. Singes allégés (animation, ciblage, particules) #153, #182	
Ce qui a été fait	**Animation.** Sur le plateau et dans la bibliothèque, les singes sont **figés**. Leur modèle animé (SkinnedMeshRenderer, recalculé à chaque image par le processeur) est remplacé par un modèle fixe dans la pose de repos. Ce modèle est calculé une fois et partagé par tous les singes du même type. Sur la carte, un singe au repos n'est plus animé quand on ne le voit pas ou quand il est à plus de 6 m du joueur. Il reste toujours animé quand il tire. **Ciblage.** La recherche de ballons réutilise la même liste (aucune allocation), et quand aucun ballon n'est à portée, le singe ne cherche plus que 10 fois par seconde au lieu de chaque image. **Particules.** L'aura n'est plus faite de particules (avant : deux systèmes par singe, jusqu'à 86 particules). C'est un seul maillage fixe, qui n'est animé que s'il est visible. Au plus 4 projectiles partent par tir.
Où	• `Scripts/Core/MonkeyView.cs:41` : `Freeze` (les singes sans `TowerAnimator`) • `Scripts/Map/TowerAnimator.cs:34` : `IdleDistance = 6f` ; les tests de visibilité et de distance lignes 101-103 • `Scripts/Map/Tower.cs:21` : `SearchDelay = 0.1f` (utilisé ligne 49) ; `MaxProjectiles = 4` ligne 13 • `Scripts/Core/Aura.cs:76` : `if (!rend.isVisible) return` ; l'aura en maillage décrite en tête du fichier
Comment vérifier	• En Play, au hub : sélectionner un singe du plateau. Son modèle a des **Mesh Renderer**, et plus de Skinned Mesh Renderer. • Sur la carte : s'éloigner à plus de 6 m d'un singe au repos, il ne bouge plus. Il se remet à bouger quand il tire. • Mode admin (**SAE** → **Mode de jeu** → **PC admin**, puis F1), 200 singes : **Profiler** → **CPU**, le temps des Animator et des scripts doit baisser par rapport à #153 (41 fps).
Ce qu'on perd	les singes du plateau ne bougent pas (ils font 5 cm de haut).

Mesures à faire

Toujours mesurer de la même façon : même endroit, même vague, on regarde dans la même direction pendant 10 secondes et on note la valeur la plus basse. Au casque, le compteur de fps est au poignet gauche. Reporter les résultats dans `docs/opti.md` et dans l'issue #35.

Mesure	Où	Concerne	Avant (#153)	Après
fps	carte, grosse vague (casque)	toutes		
fps mode admin	200 singes, grosse vague (éditeur)	1, 2, 6, 7, 8	41 fps	
SetPass calls	Stats, hub (éditeur)	2, 3, 5	141	
Triangles	Stats, mode admin (éditeur)	1, 2, 4	626 000	
GC Alloc	Profiler CPU, pendant une vague	6, 8		
Temps de physique	Profiler Physics, grosse vague	7		
Matériaux	Profiler Memory, tutoriel 3 min	3	+72 / s	

Éditeur ≠ casque. L'éditeur dit si on va dans le bon sens, mais seul le casque dit si on tient 72 fps : son GPU est celui d'un téléphone. Si une mesure est sous 72 fps, regarder dans le Profiler si c'est le CPU ou le GPU qui limite, puis ouvrir une issue avec le chiffre.